
Security Review Report
NM-0814 - JustaPermissionManager



NETHERMIND
SECURITY

(March 03, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	3
4.1	JustaPermissionManager overview	3
4.2	Permission lifecycle and EIP712 integration	3
4.3	Execution whitelisting and wildcards	4
4.4	Recurring spend limits and calendar alignment	4
4.5	Call Checkers and post-execution cleanup	4
4.6	Contract interaction diagram	4
5	Risk Rating Methodology	5
6	Issues	6
6.1	[Medium] Bypassing spend limits and revocation via unparsed increaseAllowance selector	6
6.2	[Medium] ERC721 transfers fail due to selector collisions with ERC20 spend checks	7
6.3	[Low] Allowance bypass due to epoch alignment	8
6.4	[Low] The executeBatch(...) function is incompatible with tokens that revert on zero approvals	9
6.5	[Info] An invalid spendPeriod can be created on permission end	10
6.6	[Info] Incorrect handling of calldata with length between one and three bytes	11
6.7	[Info] Missing reentrancy protection on approve function	11
7	Documentation Evaluation	12
8	Test Suite Evaluation	13
8.1	Tests Output	13
8.2	Automated Tools	14
8.2.1	AuditAgent	14
9	About Nethermind	15

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for [JustaLab's JustaPermissionManager](#) smart contract. The permissions contract enables a granular delegation and security layer designed to integrate with the **JustanAccount** smart accounts. Owners can delegate specific execution rights to **spenders** without granting full control over the underlying smart contract account. The execution rights can specify particular addresses to be allowed, and specific function selectors for each contract as well. This ensures that even with access to a contract, only intended behaviors are accessible.

The audit comprises 679 lines of the Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 7 points of attention, where two are classified as Medium, two are classified as Low and three are classified as Informational. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

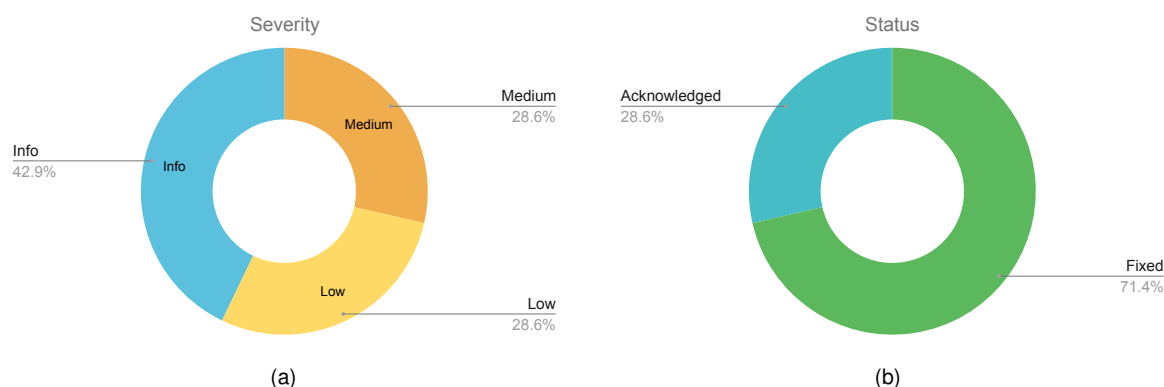


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (2), Low (2), Undetermined (0), Informational (3), Best Practices (0).
Distribution of status: Fixed (5), Acknowledged (2), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	February 13, 2026
Final Report	March 3, 2026
Initial Commit	39006de9d78b7ed132742b8b4c8314d791008fda
Final Commit	95fce1d8c88100096113974817d20d85208da17a
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	JustaPermissionManager.sol	668	356	53.3%	167	1191
2	ICallChecker.sol	11	16	145.5%	1	28
	Total	679	372	54.8%	168	1219

3 Summary of Issues

	Finding	Severity	Update
1	Bypassing spend limits and revocation via unparsed increaseAllowance selector	Medium	Fixed
2	ERC721 transfers fail due to selector collisions with ERC20 spend checks	Medium	Fixed
3	Allowance bypass due to epoch alignment	Low	Fixed
4	The executeBatch(...) function is incompatible with tokens that revert on zero approvals	Low	Acknowledged
5	An invalid spendPeriod can be created on permission end	Info	Acknowledged
6	Incorrect handling of calldata with length between one and three bytes	Info	Fixed
7	Missing reentrancy protection on approve function	Info	Fixed

4 System Overview

4.1 JustaPermissionManager overview

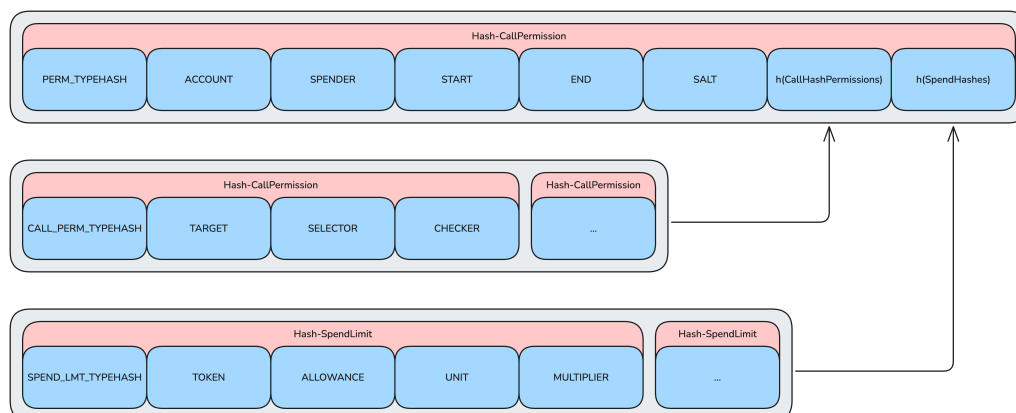
The **JustaPermissionManager** contract enables a granular delegation and security layer designed for the **JustanAccount** ecosystem. It enables account owners to delegate specific execution rights to **Spenders** without granting full control over the underlying smart contract account. The JustaPermissionManager contract is meant to be an owner within the JustanAccount contract. The Permission Manager enforces **whitelisting**, **recurring spend limits**, and **programmable execution guards**. The system is built to be compatible with both **ERC4337** (Account Abstraction) and **EIP7702** (EOA delegation) flows.

4.2 Permission lifecycle and EIP712 integration

Permissions are defined by a Permission struct that includes the account, the authorized spender, time bounds (start and end), and arrays for call and spend authorizations.

- **Approval:** An account owner must call the **approve()** function to register a permission hash. The system uses **EIP712 structured data hashing** to ensure that permission definitions are unique and signable off-chain.
- **Revocation:** Permissions can be revoked at any time by either the account owner or the spender via **revoke()** or **revokeAsSpender()**. Once revoked, the specific permission hash can never be reused, preventing replay attacks.

A diagram of the data contained within a Permission and the hashing structure is show below:



4.3 Execution whitelisting and wildcards

The core functionality revolves around the `executeBatch()` function, which allows a **Spender** to trigger multiple calls on behalf of the user's account. Every call in a batch is subject to strict authorization checks:

- **Call matching:** Each call must match an entry in the `calls` array, which specifies the target contract and the function selector.
- **Wildcards:** The protocol supports flexible delegation through constants like `ANY_TARGET` and `ANY_FN_SEL`. This allows owners to provide broad access to a specific protocol or specific functions across any contract.
- **Privilege escalation prevention:** To prevent a Spender from taking over the account, the Permission Manager strictly prohibits any delegated calls targeting the `JustaPermissionManager` contract itself or the `JustanAccount` contract directly.

4.4 Recurring spend limits and calendar alignment

A feature of the protocol is its spending tracking system. It enables **allowances** that reset over specific time intervals (e.g., 500 USDC per week).

- **Period units:** Spend limits support various time units, including Minute, Hour, Day, Week, Month, Year, and a one time Forever limit.
- **Calendar alignment:** To prevent users from "gaming" the limits, the protocol aligns periods to calendar boundaries
- **Conservative tracking:** The system tracks outflows by comparing the balance delta (before vs. after execution).

4.5 Call Checkers and post-execution cleanup

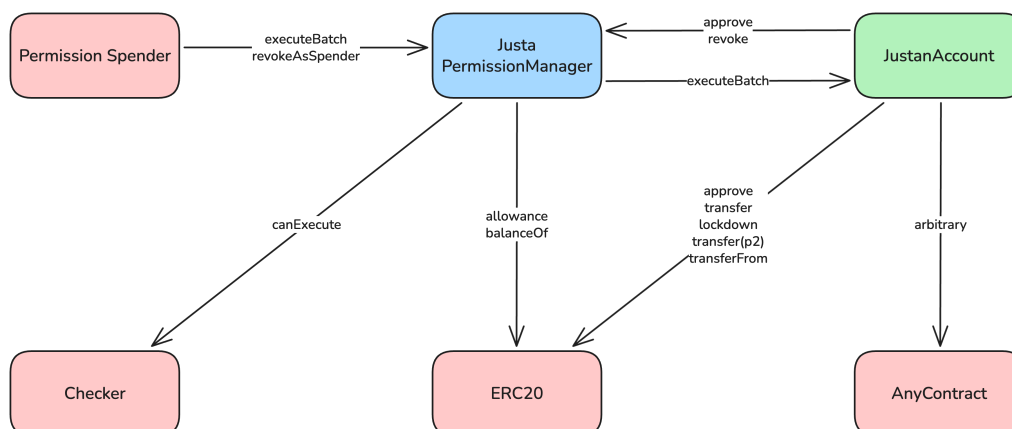
Beyond simple whitelisting, the protocol allows for programmable security through call **Checkers**.

- **Checkers:** If a `CallPermission` includes a checker address, the Permission Manager invokes `ICallChecker.canExecute()` during the execution flow. This allows external logic to validate the actual arguments of a call.
- **Approval revocation:** If a delegated call creates an ERC20 approval or a Permit2 allowance, the Permission Manager **automatically revokes** these approvals at the end of the `executeBatch` transaction. This prevents target contracts from retaining approvals that could be exploited outside the Permission Manager's spend limit tracking.

4.6 Contract interaction diagram

The protocol generally will include the following entities:

- **PermissionManager:** A singleton deployment which all accounts assign as an owner. Permissions are registered and managed here, spenders can trigger `batchExecute` calls from here which are passed to the intended account.
- **Account:** The account which a spender will act on behalf of. The account must initially approve a permission by calling the permission contract, at which point a spender can initiate transactions for the given account.
- **Spender:** A spender is given an approval by an account, at which point they can spend on behalf of the account. The spender is also able to revoke the permission given to them.
- **Checker:** An optional contract for each call that can inspect the calldata and query other contracts to ensure that the given call will behave as intended.



5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Medium] Bypassing spend limits and revocation via unparsed increaseAllowance selector

File(s): `src/JustaPermissionManager.sol`

Description: The JustaPermissionManager contract enforces spend limits and security guards during batch execution via the `executeBatch(...)` function. It achieves this by parsing the calldata of the executed calls to track token operations. Specifically, it identifies `approve(...)` calls to track the approved spenders and subsequently revokes the approval at the end of the transaction. This mechanism ensures that spenders do not retain any allowance after the batch execution is completed.

However, the calldata parser doesn't check the `increaseAllowance(...)` function selector. Because this selector is ignored during the parsing phase, the manager does not record the allowance increase or the spender's address for the post-execution revocation step.

```
1 //...
2 // `approve(address,uint256)` - 0x095ea7b3
3 // We have to revoke any new approvals after the batch, else a bad app can
4 // leave an approval to let them drain unlimited tokens after the batch.
5 if (fnSel == 0x095ea7b3) {
6     if (LibBytes.loadCalldata(data, 0x24) == 0) {
7         continue;
8     } // `amount == 0`
9     t.approvedERC20s.p(target);
10    t.approvalSpenders.p(LibBytes.loadCalldata(data, 0x04).lsbToAddress()); // `spender`
11    t.erc20s.p(target);
12    t.transferAmounts.p(LibBytes.loadCalldata(data, 0x24));
13 }
14
15 // @audit-issue The parser doesn't check for the increaseAllowance(...) selector here.
16 // Permit2 `approve(address,address,uint160,uint48)` - 0x87517c45
17 if (fnSel == 0x87517c45) {
18     // ...
19 }
```

Since calling `increaseAllowance(...)` does not immediately decrease the account's token balance, the post-execution balance difference check (`balanceDelta`) evaluates to 0. As a result, the manager treats the call as a non-spending action. This allows a potentially malicious spender to increase their allowance for any tracked token, bypassing the configured spend limits. The spender can then drain the funds from the account in a subsequent transaction.

Recommendation(s): Consider updating the calldata parsing logic within the `executeBatch(...)` function to also identify and track calls made to the `increaseAllowance(...)` function.

Status: Fixed

Update from the client: Resolved in commit [e41d3fe](#). The calldata parser in `executeBatch` now recognizes the `increaseAllowance(address,uint256)` selector (`0x39509351`). The new parsing block mirrors the existing `approve` handler.

6.2 [Medium] ERC721 transfers fail due to selector collisions with ERC20 spend checks

File(s): [src/JustaPermissionManager.sol](#)

Description: The JustaPermissionManager contract acts as a delegation layer enabling granular permissions with execution guards and spend limits. Users can execute a batch of calls using the `executeBatch(...)` function. During this execution, the manager parses the calldata for token operations to track and limit ERC20 token spending.

The `executeBatch(...)` function iterates over the execution calls and extracts the function selector. It specifically checks for the `0x23b872dd` selector, which corresponds to the ERC20 `transferFrom(...)` function. However, this selector is also identical to the ERC721 `transferFrom(...)` function. When a user attempts to transfer an NFT using this function, the parser incorrectly treats the third parameter (the `tokenId`) as an ERC20 amount.

```

1 // ...
2 if (fnSel == 0x23b872dd) {
3     address from = LibBytes.loadCalldata(data, 0x04).lsbToAddress();
4     if (LibBytes.loadCalldata(data, 0x44) == 0) {
5         continue;
6     } // `amount == 0`
7     // @audit-issue Treats ERC721 transferFrom as an ERC20, interpreting tokenId as an amount.
8     if (from == permission.account) {
9         t.erc20s.p(target);
10        t.transferAmounts.p(LibBytes.loadCalldata(data, 0x44)); // `amount`
11    }
12 }
13 // ...
14
```

Later in the execution flow, the contract attempts to record the calculated spend by calling the `_checkAndIncrementSpend(...)` function. The `approve(...)` function, which is responsible for setting up permissions, explicitly forbids configuring `SpendLimit` entries for ERC721 tokens. If a user tries to create a spend limit for an NFT, it reverts with `JustaPermissionManager_ERC721TokenNotSupported`.

```

1 // ...
2 if (permission.spends[i].token != NATIVE_TOKEN) {
3     // @audit Spend limits for ERC721 tokens are explicitly forbidden during approval.
4     if (ERC165Checker.supportsInterface(permission.spends[i].token, type(IERC721).interfaceId)) {
5         revert JustaPermissionManager_ERC721TokenNotSupported(permission.spends[i].token);
6     }
7     // ...
8
```

Because no valid spend limits can exist for ERC721 tokens, the `_checkAndIncrementSpend(...)` function will fail to find an entry for the NFT contract and will systematically revert with `JustaPermissionManager_NoSpendPermissions`. This means that users are prevented from transferring ERC721 tokens.

Recommendation(s): Consider distinguishing between ERC20 and ERC721 target contracts before parsing the `transferFrom(...)` calldata.

Status: Fixed

Update from the client: Resolved in commit [fc7111b](#). When the `executeBatch` calldata parser encounters the `0x23b872dd` (`transferFrom`) selector, it now performs an ERC-165 `supportsInterface` check on the target contract before parsing the calldata. If the target supports the IERC721 interface, the parser skips the call entirely via `continue`, avoiding the misinterpretation of the NFT `tokenId` as an ERC-20 amount.

6.3 [Low] Allowance bypass due to epoch alignment

File(s): `src/JustaPermissionManager.sol`

Description: The JustaPermissionManager implements spend limits using the PeriodUnit enum. To prevent users from manipulating period boundaries, the contract aligns Day-based periods to the Unix Epoch (Jan 1, 1970, which was a Thursday). The `startOfSpendPeriod(...)` function calculates the start of a period by performing modulo arithmetic against this absolute timestamp.

```

1 // ...
2 if (unit == PeriodUnit.Day) {
3     // Align to day boundary, then find the correct multiplier window
4     uint256 baseStart = Math.rawMul(Math.rawDiv(unixTimestamp, 86_400), 86_400);
5     uint256 periodDuration = 86_400 * uint256(multiplier);
6     // @audit-issue Aligns periods to absolute Unix Epoch buckets.
7     return Math.rawMul(Math.rawDiv(baseStart, periodDuration), periodDuration);
8 }
9 // ...
10

```

This alignment strategy creates an extra period for the spender. For example, if a user creates a 5 day permission on a Wednesday, the current epoch aligned cycle will end on the following day, Thursday. The spender can consume their allowance on Wednesday and as soon as the clock ticks over to Thursday, the `_getCurrentPeriod(...)` logic identifies a new period start, effectively resetting the limit and granting a fresh allowance.

Consequently, for a permission intended to span exactly N days, the spender may actually receive allowances for $N+1$ periods if the start time is misaligned with the epoch. The shorter the time span, the bigger the impact. In the case of short-term permissions (e.g., a 1-week pass), this results in a 100% overspend capability, as the spender gets access to two full allowances within the intended duration. However, if the spender would have allowances for 100 cycles, spending 101 allowances, diminishes the impact (only 1% overspend).

Recommendation(s): Consider revisiting the logic for how the start of the period is computed or acknowledge the behavior and design limitations.

Status: Fixed

Update from the client: Resolved in commit [53b9021](#). All period types (Minute, Hour, Day, Week, Month) are now aligned to the permission's own start timestamp instead of the Unix Epoch. The `startOfSpendPeriod` function now accepts a `permissionStart` parameter and computes period boundaries as:

- Fixed units (Minute, Hour, Day, Week):
 $periodStart = permissionStart + \text{floor}((elapsed / duration) * duration)$, where $elapsed = currentTimestamp - permissionStart$.
- Month: Uses `DateTimelib.addMonths(permissionStart, periodIndex * multiplier)` for calendar-correct day-clamping.

This eliminates the extra period at the beginning of a permission. Since the first period always starts exactly at `permissionStart`, a spender can no longer exploit a misalignment between the permission creation time and an epoch-derived boundary to gain a fresh allowance prematurely. The `_getCurrentPeriod` function was also simplified to remove the special-case handling for permissions that started mid-period, as that scenario can no longer occur with permission-aligned periods.

6.4 [Low] The executeBatch(...) function is incompatible with tokens that revert on zero approvals

File(s): `src/JustaPermissionManager.sol`

Description: The JustaPermissionManager contract allows users to execute multiple calls within a single transaction using the `executeBatch(...)` function. When a user approves an ERC20 token for a spender, the manager tracks this action during the calldata parsing phase. After all calls are executed, the manager automatically revokes all recorded approvals by resetting the allowance to 0. This is a security measure designed to prevent spenders from retaining access to the account's tokens after the batch is completed.

However, the `executeBatch(...)` function assumes that all ERC20 tokens will successfully execute an `approve(...)` call when the amount is set to 0. There is a known category of non-standard ERC20 tokens (weird tokens), such as the older BNB token on Ethereum (0xB8c77482e45F1F44dE1745F52C74426C631bDD52), that revert when their `approve(...)` function is called with a 0 value.

```
1 // ...
2 uint256 approvedLength = t.approvedERC20s.length();
3 if (approvedLength > 0) {
4     BaseAccount.Call[] memory revokeCalls = new BaseAccount.Call[](approvedLength);
5     for (uint256 i; i < approvedLength; ++i) {
6         revokeCalls[i] = BaseAccount.Call({
7             target: t.approvedERC20s.getAddress(i),
8             value: 0,
9             // @audit-issue Encodes a 0-value approval, which reverts for certain tokens.
10            data: abi.encodeWithSelector(IERC20.approve.selector, t.approvalSpenders.getAddress(i), 0)
11        });
12    }
13    _executeBatch(permission.account, revokeCalls);
14 // ...
```

If a user includes such a token within their execution batch, the transaction will revert. This leads to a DOS for users attempting to interact with such tokens through the permission manager.

Recommendation(s): Consider acknowledging the incompatibility of the contract with such tokens.

Status: Acknowledged

Update from the client: We accept this limitation. Tokens such as USDT require the allowance to be set to zero before a new approval can be granted. Setting the residual allowance to 1 wei would break compatibility with USDT and similar tokens that enforce this pattern, which represent a significantly larger and more critical category than tokens that revert on zero-value approvals. We therefore retain the zero-value revocation to ensure correct behavior with standard tokens like USDT.

6.5 [Info] An invalid spendPeriod can be created on permission end

File(s): src/JustaPermissionManager.sol

Description: The function `_getCurrentPeriod` is used to load data on the current spend period, with the data consisting of a period start, end, and amount spent. It is possible for a "phantom" spendPeriod with a duration of zero to be created when this function is called with a `block.timestamp` that matches the `permission.end`. This can only happen when the entry in `_lastUpdatedPeriod` is not still active, since a stored period that is still active will lead to an early return.

When calculating the `periodStart` using `startOfSpendPeriod(...)`, a spend period may align with the end of a permission, making the `periodStart` equal to `permission.end`. Since this is the final spend period, we will enter the `else` branch (shown below) and add the duration to calculate the `periodEnd`.

```

1  function _getCurrentPeriod(...) ... {
2      // ...
3
4      // @audit Most recent stored period is not active
5      if (lastPeriodExists && lastPeriodStillActive) {
6          return lastUpdatedPeriod;
7      }
8
9      // ...
10     // @audit Period start can be permission.end
11     uint48 periodStart = uint48(startOfSpendPeriod(...));
12     uint48 periodEnd;
13
14     if (...) {} else {
15         // @audit Add duration to the start of period
16         uint48 duration = _periodDuration(...);
17         uint256 calculatedEnd = uint256(periodStart) + uint256(duration);
18         // ...
19         periodEnd = uint48(calculatedEnd);
20     }
21
22     // @audit Cut the duration off, setting period end to permission.end
23     //      When start is also permission.end you have zero duration period
24     if (periodEnd > permissionEnd) {
25         periodEnd = permissionEnd;
26     }
27     // ...
28 }

```

When the check for the period end exceeding the permission end is executed, the period end is cut down to `permission.end`. This leads to both the `periodStart` and `periodEnd` being equal, so a zero duration period is created. This does not allow for a double spend however, since a prerequisite for this bug to occur is to have not used the previous (final) spend period. Effectively if this bug were used, it would lead to the same amount of spend but just on the extra zero duration period.

Status: Acknowledged

Update from the client: As noted in the finding, this edge case does not enable a double spend, it can only occur when the final spend period was unused, and any spend recorded on the phantom zero-duration period is bounded by the same limit that would have applied to the preceding period. The impact is cosmetic (a zero-duration period entry in storage) with no economic consequence. We accept this behavior as-is.

6.6 [Info] Incorrect handling of calldata with length between one and three bytes

File(s): [src/JustaPermissionManager.sol](#)

Description: The JustaPermissionManager contract utilizes the `executeBatch(...)` function to iterate over and execute a batch of calls. During this process, the contract extracts the function selector from the provided calldata to enforce authorization rules against the user's permissions.

The current parsing logic assumes that any calldata with a length strictly less than 4 bytes is entirely empty, and subsequently assigns it the `EMPTY_CALLDATA_FN_SEL` selector. However, this assumption is wrong because it is possible to submit transactions with a calldata payload of 1, 2, or 3 bytes.

```
1 // Extract function selector (use EMPTY_CALLDATA_FN_SEL for empty calldata)
2 // @audit-issue Assumes lengths of one, two, and three bytes are empty calldata.
3 bytes4 selector = calls[i].data.length >= 4 ?
4     bytes4(LibBytes.loadCalldata(calls[i].data, 0x00)) : EMPTY_CALLDATA_FN_SEL;
5 // ...
6 }
```

While there is no immediate exploit vector since the EVM will revert on such malformed calls during the execution step, treating a partially populated payload as completely empty represents a logical error.

Recommendation(s): Consider revising the length validation logic. The function should handle cases where the calldata length is exactly '0' bytes and revert on payloads that fall into the invalid length range of '1' to '3' bytes.

Status: Fixed

Update from the client: Resolved in commit [77f27eb](#). The `executeBatch` function now explicitly validates calldata length before extracting the function selector. A new custom error `JustaPermissionManager_InvalidCalldataLength` is introduced, and calldata with a length between 1 and 3 bytes is rejected with a revert. The zero-length case continues to be handled correctly via `EMPTY_CALLDATA_FN_SEL`, and the existing selector extraction now reuses the cached `dataLength` variable.

6.7 [Info] Missing reentrancy protection on approve function

File(s): [src/JustaPermissionManager.sol](#)

Description: The README documentation states that reentrancy protection is applied to all state modifying functions, however the function `approve` does not have reentrancy protection. Adding the `nonReentrant` modifier will not only align the logic with the documentation, but will act as an additional security measure against privilege escalation attempts that involve attempting to trigger a call to `approve` through a `batchExecute` call.

Recommendation(s): Consider adding the `nonReentrant` modifier to the `approve` function.

Status: Fixed

Update from the client: Resolved in commit [859f4bf](#). The `nonReentrant` modifier has been added to the `approve` function, aligning the implementation with the documented invariant that all state-modifying functions have reentrancy protection.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about JustaPermissionManager's documentation

JustaLab's team provided an overview of the main system components during the kick-off call with a detailed explanation of the intended functionalities and use cases of the system. Moreover, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

```
> forge test

Ran 1 test for test/integration/TestBatchExecutionFlow.t.sol:TestBatchExecutionFlow
[PASS] test_ShouldExecuteBatchTransfersThroughPermission(...) (runs: 256)

Ran 2 tests for test/integration/TestPermissionLifecycleFlow.t.sol:TestPermissionLifecycleFlow
[PASS] test_ShouldAllowSpenderToRevokePermission(...) (runs: 256)
[PASS] test_ShouldFollowPermissionLifecycle(...) (runs: 256)

Ran 4 tests for test/integration/TestSpendLimitEnforcementFlow.t.sol:TestSpendLimitEnforcementFlow
[PASS] test_ShouldEnforceSpendLimitAcrossBatches(...) (runs: 256)
[PASS] test_ShouldEnforceSpendLimitWithStartZeroAndForeverPeriod(...) (runs: 256)
[PASS] test_ShouldRevertERC20TransferWithEmptySpends(...) (runs: 256)
[PASS] test_ShouldRevertNativeETHTransferWithEmptySpends(...) (runs: 256)

Ran 8 tests for test/integration/TestCallCheckerFlow.t.sol:TestCallCheckerFlow
[PASS] test_MultipleCheckers_BothApprove_RealTokenTransfer(...) (runs: 256)
[PASS] test_ShouldExecuteWhenCheckerApproves(...) (runs: 256)
[PASS] test_ShouldPassCorrectParamsToChecker(...) (runs: 256)
[PASS] test_ShouldRevertIfCheckerApprovesButSpendLimitExceeded(...) (runs: 256)
[PASS] test_ShouldRevertIfWithinSpendLimitButCheckerRejects(...) (runs: 256)
[PASS] test_ShouldRevertWhenCheckerRejects(...) (runs: 256)
[PASS] test_ShouldSucceedWhenBothCheckerAndSpendLimitPass(...) (runs: 256)
[PASS] test_WildcardCannotBeBypassedByNoCheckerSpecific(...) (runs: 256)

Ran 1 test for test/integration/TestPermissionManagerFlow.t.sol:TestPermissionManagerFlow
[PASS] test_ShouldExecuteERC20TransferThroughPermission(...) (runs: 256)

Ran 5 tests for test/integration/Test7702BatchSetupFlow.t.sol:Test7702BatchSetupFlow
[PASS] test_ShouldBatchAddMultipleOwnersAndApprove(...) (runs: 256)
[PASS] test_ShouldBatchAddOwnerThenApprove(...) (runs: 256)
[PASS] test_ShouldBatchApproveThenAddOwner(...) (runs: 256)
[PASS] test_ShouldFailIfManagerNotAddedAsOwner(...) (runs: 256)
[PASS] test_ShouldWorkWithCrossChainDelegation(...) (runs: 256)

Ran 68 tests for test/unit/TestWriteFunctions.t.sol:TestWriteFunctions
[PASS] test_Approve_RevertIfAlreadyRevoked(...) (runs: 256)
[PASS] test_Approve_RevertIfCallSelectorIsZero(...) (runs: 256)
[PASS] test_Approve_RevertIfCallTargetIsZero(...) (runs: 256)
[PASS] test_Approve_RevertIfCallTargetsAccount(...) (runs: 256)
[PASS] test_Approve_RevertIfCallTargetsSelf(...) (runs: 256)
[PASS] test_Approve_RevertIfCheckerHasNoCode(...) (runs: 256)
[PASS] test_Approve_RevertIfCheckerTargetsAccount(...) (runs: 256)
[PASS] test_Approve_RevertIfCheckerTargetsSelf(...) (runs: 256)
[PASS] test_Approve_RevertIfDuplicateSpendLimits(...) (runs: 256)
[PASS] test_Approve_RevertIfEmptyCallsArray(...) (runs: 256)
[PASS] test_Approve_RevertIfNotAccountOwner(...) (runs: 256)
[PASS] test_Approve_RevertIfSpendAllowanceIsZero(...) (runs: 256)
[PASS] test_Approve_RevertIfSpendMultiplierIsZero(...) (runs: 256)
[PASS] test_Approve_RevertIfSpendTokenIsERC1155(...) (runs: 256)
[PASS] test_Approve_RevertIfSpendTokenIsERC721(...) (runs: 256)
[PASS] test_Approve_RevertIfSpendTokenIsZero(...) (runs: 256)
[PASS] test_Approve_RevertIfStartAfterEnd(...) (runs: 256)
[PASS] test_Approve_RevertIfStartEqualsEnd(...) (runs: 256)
[PASS] test_Approve_RevertIfZeroSpender(bytes4,uint160,uint8,uint8) (runs: 256)
[PASS] test_Approve_ShouldApprovePermissionAndEmitEvent(...) (runs: 256)
[PASS] test_Approve_ShouldReturnTrueIfAlreadyApproved(...) (runs: 256)
[PASS] test_DoubleSpendAllowances() (gas: 358483)
[PASS] test_ExecuteBatch_MultipleMatchingCheckers_AnyRejectsReverts(...) (runs: 256)
[PASS] test_ExecuteBatch_MultipleMatchingCheckers_BothApprove(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfAfterEnd(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfApprovalRevocationFailed(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfBeforeStart(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfCallTargetsAccount(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfCallTargetsSelf(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfCheckerRejectsCall(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfExceededSpendLimit(...) (runs: 256)
```

```
[PASS] test_ExecuteBatch_RevertIfNoSpendPermissionForToken(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfNotApproved(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfNotSpender(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfRevoked(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfSpendValueOverflow(...) (runs: 256)
[PASS] test_ExecuteBatch_RevertIfUnauthorizedCall(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldAccumulateSpendInSamePeriod(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldAllowAnySelectorWildcard(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldAllowAnyTargetWildcard(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldAllowEmptyCallldataWithSpecialSelector(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldAllowMixedCheckerCalls(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldCheckAllCallsWithCheckersBeforeExecution(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldCreateNewPeriodOnFirstSpend(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldEmitCallsExecutedEvent(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldExecuteMultipleCalls(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldExecuteSingleCall(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldPassCorrectParamsToChecker(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldResetSpendOnNewPeriod(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldRevokeERC20ApprovalsAfterExecution(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldRevokePermit2ApprovalsAfterExecution(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldSucceedWithCheckerApproval(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldTrackERC20Approve(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldTrackERC20Transfer(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldTrackERC20TransferFrom(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldTrackNativeTokenSpend(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldTrackPermit2Approve(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldTrackSelfToSelfTransfer(...) (runs: 256)
[PASS] test_ExecuteBatch_ShouldWorkWithoutChecker(...) (runs: 256)
[PASS] test_ExecuteBatch_WildcardCheckerRunsWhenSpecificHasNoChecker(...) (runs: 256)
[PASS] test_RevokeAsSpender_RevertIfNotApproved(...) (runs: 256)
[PASS] test_RevokeAsSpender_RevertIfNotSpender(...) (runs: 256)
[PASS] test_RevokeAsSpender_ShouldBeIdempotent(...) (runs: 256)
[PASS] test_RevokeAsSpender_ShouldRevokeAndEmitEvent(...) (runs: 256)
[PASS] test_Revoke_RevertIfNotAccountOwner(...) (runs: 256)
[PASS] test_Revoke_RevertIfNotApproved(...) (runs: 256)
[PASS] test_Revoke_ShouldBeIdempotent(...) (runs: 256)
[PASS] test_Revoke_ShouldRevokeAndEmitEvent(...) (runs: 256)

Ran 22 tests for test/unit/TestReadFunctions.t.sol:TestReadFunctions
[PASS] test_GetCurrentPeriod_ShouldReturnValidPeriod(...) (runs: 256)
[PASS] test_GetCurrentPeriod_ShouldRevertIfAfterPermissionEnd(...) (runs: 256)
[PASS] test_GetCurrentPeriod_ShouldRevertIfBeforePermissionStart(...) (runs: 256)
[PASS] test_GetHash_ShouldReturnDifferentHashForDifferentSalt(...) (runs: 256)
[PASS] test_GetHash_ShouldReturnDifferentHashForDifferentSpender(...) (runs: 256)
[PASS] test_GetHash_ShouldReturnNonZeroHash(...) (runs: 256)
[PASS] test_GetHash_ShouldReturnSameHashForSamePermission(...) (runs: 256)
[PASS] test_GetLastUpdatedPeriod_ShouldReturnZeroEvenAfterApproval(...) (runs: 256)
[PASS] test_GetLastUpdatedPeriod_ShouldReturnZeroForUnusedPermission(...) (runs: 256)
[PASS] test_IsApproved_ShouldReturnFalseForUnapprovedPermission(...) (runs: 256)
[PASS] test_IsApproved_ShouldReturnTrueEvenAfterRevoked(...) (runs: 256)
[PASS] test_IsApproved_ShouldReturnTrueForApprovedPermission(...) (runs: 256)
[PASS] test_IsRevoked_ShouldReturnFalseForNonRevokedPermission(...) (runs: 256)
[PASS] test_IsRevoked_ShouldReturnTrueForRevokedPermission(...) (runs: 256)
[PASS] test_ShouldReturnCorrectAnyFnSel() (gas: 6493)
[PASS] test_ShouldReturnCorrectAnyTarget() (gas: 6585)
[PASS] test_ShouldReturnCorrectCallPermissionTypehash() (gas: 7182)
[PASS] test_ShouldReturnCorrectEmptyCallldataFnSel() (gas: 6824)
[PASS] test_ShouldReturnCorrectNativeToken() (gas: 5497)
[PASS] test_ShouldReturnCorrectPermissionTypehash() (gas: 6984)
[PASS] test_ShouldReturnCorrectPermit2...() (gas: 6916)
[PASS] test_ShouldReturnCorrectSpendLimitTypehash() (gas: 5994)

Ran 8 test suites in 885.69ms (4.69s CPU time): 111 tests passed, 0 failed, 0 skipped (111 total tests)
```

8.2 Automated Tools

8.2.1 AuditAgent

The AuditAgent is an AI-powered smart contract auditing tool that analyses code, detects vulnerabilities, and provides actionable fixes. It accelerates the security analysis process, complementing human expertise with advanced AI models to deliver efficient and comprehensive smart contract audits. Available at <https://app.auditagent.nethermind.io>.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.